



TED UNIVERSITY

CMPE 492 / SENG 492 Senior Project II

Center of Gravity Detection, Information, and Balancing System in Aircraft

Test Plan Report

10.04.2026

Team Members:

Selin Göç, Computer Engineering

Elif Sude Memiş, Software Engineering

Sude İpekci, Computer Engineering

Yasin Anber, Computer Engineering

Supervisor:

Prof. Dr. Hakkı Gökhan İLK

Jury Members:

Dr. Ali BERKOL

Dr. Ayşe Yasemin SEYDİM

Table of Contents

1. Introduction	3
1.1 Purpose of the Document	3
1.2 Scope of the System	3
2. Test Objectives	3
3. Features to be Tested	4
4. Testing Methodology	5
4.1 Unit Testing	5
4.2 Integration Testing	6
4.3 System Testing	6
4.4 Performance Testing	6
4.5 User Acceptance Testing (UAT)	7
4.6 Beta Testing	7
5. Test Environment	7
6. Test Cases	8
6.1 Unit Test Case (TC-01 to TC-07)	9
6.2 Integration Test Cases (TC-08 to TC-11)	13
6.3 System Test Cases (TC-12 to TC-13, TC-17 to TC-18)	16
6.4 Performance Test Cases (TC-14 to TC-16)	19
6.5 User Acceptance Test Case (TC-19)	21
6.6 Beta Test Case (TC-20)	21
7. Test Schedule	22
8. Roles and Responsibilities	23
9. Control Procedures	24
9.1 Test Execution Protocol	24
9.2 Defect Management	24
9.3 Version Control and Configuration Management	24
10. Risks and Mitigation	25
11. Conclusion	26
12. References	27

1. Introduction

1.1 Purpose of the Document

This test plan document defines the scope, approach, resources, and schedule of the testing activities for the Aircraft Center of Gravity (CG) Detection and Balancing System. Its primary objective is to systematically identify and evaluate the components and functionalities of the system to ensure correctness, reliability, and safety. The document outlines the features to be tested, the testing methodologies to be employed and the responsibilities of the personnel involved in the testing process. Furthermore, it establishes a structured framework for verifying that the system performs accurately under various operational conditions and meets the specified functional and performance requirements. Through this comprehensive testing strategy, the document aims to support the development of a robust and efficient solution for aircraft balance management [11][12].

1.2 Scope of the System

The Aircraft Center of Gravity (CG) Detection and Balancing System is designed to measure, analyze, and correct the balance of an aircraft using an integrated hardware–software architecture. The system incorporates load cell sensors to accurately measure weight distribution across the aircraft structure, with signal amplification and analog-to-digital conversion performed by the HX711 module [1][3][4]. The processed data is handled by an Arduino-based microcontroller, which executes the CG calculation algorithm to determine the precise center of gravity coordinates[7][8][9]. Based on the computed results, a servo motor-driven balancing mechanism adjusts a counterweight to correct any detected imbalance. Additionally, a user interface is provided to monitor system status, visualize CG information and support real-time interaction. Overall, the system operates as a closed-loop solution that combines sensing, computation, actuation, and visualization to ensure safe and efficient aircraft balance management.

2. Test Objectives

The overarching goal of the testing effort is to provide documented evidence that the Aircraft CG Detection and Balancing System meets its specified functional and non-functional requirements. The specific objectives of this test plan are as follows:

- Verify that each individual hardware component (load cells, HX711 modules, servo motors) operates correctly within its specified parameters and tolerance ranges [1][3].
- Confirm that the CG calculation algorithm produces results that are accurate to within ± 3 mm of manually computed reference values under all tested load distributions [2].
- Validate the correctness and reliability of inter-subsystem communication, including HX711-to-Arduino synchronous serial communication and Arduino-to-PC UART data transmission [5][6].
- Assess the end-to-end balancing cycle from imbalance detection through control computation to physical counterweight actuation for correctness, speed and repeatability.
- Evaluate system performance characteristics including control loop response time, data transmission throughput, and long-term operational stability.
- Verify the implementation of safety mechanisms including divide-by-zero protection, mechanical axis boundary enforcement, sensor failure detection, and emergency halt functionality.
- Confirm that the user interface (CGVisualizerApp) provides accurate, timely and interpretable visual feedback suitable for use by ground operations personnel.
- Identify, document and facilitate the resolution of any defects, inconsistencies or performance bottlenecks discovered during testing prior to final demonstration.

3. Features to be Tested

Table 1 enumerates the primary system features subject to testing, along with a brief description of each and the relevant system components involved.

Table 1. System Features Subject to Testing

#	Feature	Description	Component(s)
1	Load Cell Sensing	Measurement of vertical force at each aircraft landing support point using strain gauge load cells, including calibration accuracy and noise behavior.	Load Cells, Wheatstone Bridge
2	Signal Amplification & ADC Conversion	Amplification of microvolt-level load cell signals and 24-bit analog-to-digital conversion by the HX711 module, including stability and resolution.	HX711 Module
3	Digital Filtering (LPF & Deadzone)	Application of Low-Pass Filter with 0.05 Hz cutoff and ± 1.0 g deadzone logic to suppress noise and micro-fluctuations in sensor readings.	SensorModule (Arduino firmware)
4	CG Calculation Algorithm	Computation of X and Y center of gravity coordinates using moment equilibrium equations, including divide-by-zero protection.	CGCalculationModule
5	Control Decision Logic	Evaluation of CG deviation against balance tolerance, computation of counterweight target position using mass-moment equations, and deadband enforcement.	ControlModule
6	Servo Motor Actuation	PWM-controlled servo motor movement along X and Y axes via lead screw mechanisms, including position accuracy and homing calibration.	MotorControlModule, Servo Motors, Lead Screws
7	UART Serial Communication	Real-time transmission of weight, CG coordinates, and system status from Arduino to PC at 9600 baud.	Arduino UART, USB Interface
8	User Interface Visualization	Real-time 2D graphical display of CG marker position, color-coded stability	CGVisualizerApp (Python/Tkinter)

#	Feature	Description	Component(s)
		indicator, and weight data in the CGVisualizerApp.	
9	Fault Handling & Safety Mechanisms	Detection of sensor anomalies, mechanical boundary enforcement, emergency halt capability, and graceful error handling.	All subsystems
10	End-to-End Balancing Cycle	Complete autonomous balancing workflow from imbalance detection to stable correction, including timing and repeatability.	Complete System

4. Testing Methodology

The testing of the Aircraft Center of Gravity Detection and Balancing System follows a structured, multi-level approach consistent with established software and systems engineering practices. Testing progresses through successive levels, from isolated component verification to full system validation in a simulated operational environment, ensuring that defects are identified at the earliest possible stage and that integrated system behavior meets all requirements.

4.1 Unit Testing

Unit testing is the first level of testing, targeting individual hardware components and software modules in isolation [11][12]. Each functional unit is tested independently to verify that it behaves correctly according to its specification before it is integrated with other components. For this project, unit testing covers the following items:

- **Load Cell Accuracy:** Each of the three load cells is individually loaded with calibration weights and the output is compared against known reference values to verify measurement accuracy within the $\pm 2\%$ tolerance requirement.
- **HX711 ADC Stability:** The analog-to-digital conversion output of each HX711 module is assessed for noise level, repeatability, and resolution under a constant applied load.
- **SensorModule Firmware (LPF and Deadzone):** The C/C++ firmware functions for low-pass filtering and deadzone suppression are tested by introducing known noise patterns and sub-threshold vibrations.
- **CGCalculationModule Firmware:** The CG computation algorithm is validated in isolation using pre-computed input data sets with known expected outputs.
- **MotorControlModule:** Servo motor response to specific PWM commands is measured to verify positional accuracy and the homing sequence is executed to verify reference frame establishment.

Unit tests are executed using the Arduino IDE Serial Monitor for embedded firmware modules, and Python scripts for independent validation of CG calculation logic [5][6]. All results are logged with timestamps.

4.2 Integration Testing

Integration testing verifies the correct interaction between subsystems when they are combined. It detects interface defects, data format mismatches, timing issues and communication errors that cannot be identified during unit testing [11][12]. The integration testing strategy follows a bottom-up approach, beginning with the lowest-level hardware interfaces and progressively expanding to encompass higher-level software integrations.

- **Sensor-to-ADC Integration:** The electrical interface between each load cell and its HX711 module is tested to confirm correct signal amplification and bit-correct digital output.
- **HX711-to-Arduino Integration:** The synchronous two-wire serial communication (DT/SCK) between all three HX711 modules and the Arduino is tested for correct 24-bit data transfer, channel independence, and timing compliance [5][6].
- **Arduino-to-PC Serial Integration:** The UART transmission of formatted data packets from the Arduino to the CGVisualizerApp is tested for data integrity, packet format compliance, and transmission rate.
- **CG Calculation to Control Module Integration:** The output of the CGCalculationModule is verified as a correct input to the ControlModule, ensuring accurate propagation of CG coordinates for target position computation.
- **Control Module to Motor Control Integration:** The target position output of the ControlModule is verified to correctly drive the MotorControlModule, resulting in appropriate physical counterweight displacement.

4.3 System Testing

System testing evaluates the behavior of the fully integrated system as a whole against the complete set of specified requirements [11][12]. At this level, the system is treated as a black box and tested by applying realistic inputs and observing system-level outputs, without specific concern for internal module boundaries. System test scenarios include:

- **End-to-End Balancing Cycle:** The complete autonomous workflow from weight application to balanced state is executed and verified for correctness and completeness.
- **Multiple Load Distribution Scenarios:** Various asymmetric loading configurations are applied to verify correct CG computation and appropriate balancing response across the full range of expected operational scenarios.
- **Fault Injection Testing:** Hardware faults such as sensor disconnection and out-of-bounds actuation commands are deliberately introduced to verify safe error handling and system protection mechanisms.
- **Mechanical Boundary Testing:** Commanded positions exceeding defined axisLimits are used to verify that motor travel constraints are correctly enforced.

4.4 Performance Testing

Performance testing assesses the system's temporal and quantitative behavior to ensure it meets real-time operational requirements [11][12]. Key performance metrics include the control loop response time (time from imbalance detection to stable correction), sensor data sampling rate, serial communication throughput, and long-term operational stability. The performance tests include:

- **Response Time Measurement:** The elapsed time from imbalance introduction to system-declared stable state is measured across multiple trials to compute mean and worst-case response times.

- Continuous Operation / Endurance Test: The system is operated continuously under a constant load for 60 minutes to detect sensor drift, memory issues, or mechanical degradation.
- Dynamic Load Change Response: The system's ability to handle sequential, rapid load changes and achieve balance after each transition is evaluated.

4.5 User Acceptance Testing (UAT)

User acceptance testing is conducted to verify that the system meets the expectations and usability requirements of its intended operators such as aircraft technicians, maintenance engineers and ground operations personnel [11][12]. UAT focuses on the CGVisualizerApp interface and the overall system interaction from a non-developer perspective. Acceptance criteria include correctness of balance status identification, responsiveness of the display, clarity of CG coordinate reporting, and ease of interpretation. A simulated operator evaluates the interface under multiple load scenarios without requiring knowledge of the underlying software architecture

4.6 Beta Testing

Beta testing represents the highest-level validation phase, conducted in a simulated near-operational environment using the aircraft prototype model [11][12]. This phase evaluates the complete system's performance across realistic pre-flight CG verification scenarios that mirror actual use cases in the aviation maintenance context. A series of loading configurations representing empty aircraft, forward-heavy, rear-heavy, and laterally imbalanced conditions are applied to the aircraft model prototype mounted on the sensing platform. The system's computed CG values are compared against theoretical reference values and the automatic balancing response is evaluated for each scenario. Successful beta testing demonstrates that the system is ready for its intended operational purpose.

5. Test Environment

All testing activities are conducted in the TED University Computer Engineering and Electronics Laboratory. The test environment comprises the following hardware and software components:

5.1 Hardware Environment

Table 2. Hardware Environment

Component	Specification	Purpose
Arduino Uno (ATmega328P)	16 MHz, 14 digital I/O pins, 6 analog inputs	Embedded control unit
Load Cells (×3)	Strain gauge full-bridge, max capacity 5 kg each	Weight measurement at landing points
HX711 ADC Modules (×3)	24-bit resolution, 80 SPS, PGA gain up to 128×	Signal amplification and ADC conversion
Servo Motors (×2)	360° rotation, PWM control, 5V operation	X and Y axis counterweight actuation

Component	Specification	Purpose
Lead Screw Assemblies (×2)	Standard pitch, mounted on orthogonal axes	Linear motion from servo rotation
Counterweight Platform	3D-printed, calibrated mass	Moving balancing mass for CG correction
Power Supply	5V regulated, USB-based for Arduino	System power
Calibration Weights	50 g, 100 g, 200 g, 300 g, 500 g sets	Reference weights for sensor calibration
PC Workstation	Windows 10/11, USB port available	Serial monitoring and visualization host

Software Environment

Table 3. Software Environment

Software Tool	Version / Details	Purpose
Arduino IDE	v2.x or later	Firmware development, upload, and Serial Monitor
Python	v3.9 or later	CGVisualizerApp runtime environment
Tkinter Library	Included with Python standard library	Graphical user interface for visualization
PySerial Library	v3.5 or later	Serial communication between Python app and Arduino
HX711 Arduino Library	Bogde/HX711 v0.7.x	Hardware abstraction for HX711 communication
Serial Terminal (optional)	PuTTY or Arduino Serial Monitor	Raw data logging during testing

6. Test Cases

The following section presents the complete set of test cases designed for this system. All test cases are numbered up to 20 as TC-XX and includes all information necessary for reproducible execution: preconditions, inputs, step-by-step procedure, expected outcomes and pass/fail criteria. Test cases are organized by the testing phase in which they are primarily executed.

6.1 Unit Test Case (TC-01 to TC-07)

Table 4. TC-01: Load Cell Calibration and Weight Accuracy

Test ID	TC-01
Test Name	Load Cell Calibration and Weight Accuracy
Description	Verify that each of the three load cells, when individually loaded with a known calibration weight, produces a measured value within the acceptable tolerance range of $\pm 2\%$ of the actual applied weight.
Preconditions	System powered on, all three HX711 modules connected, calibration factors set (393.5, -409.8, 383.7) and operation completed successfully.
Test Input	Calibration weights: 50 g, 100 g, 200 g, 500 g applied individually to each of the three load cells (Front, Right, Left).
Test Steps	1. Place known weight on Load Cell 1 (Front). 2. Record HX711 digital output via Arduino Serial Monitor. 3. Compare shown value to actual weight. 4. Repeat for Load Cell 2 (Right) and Load Cell 3 (Left). 5. Repeat for all calibration weight values.
Expected Result	All recorded values fall within $\pm 2\%$ of the known applied weight. No sensor returns a reading of 0 g when loaded.
Pass Criteria	All 12 measurements (3 sensors $\times$ 4 weights) pass within $\pm 2\%$ tolerance.
Responsible	Sude İpekci, Yasin Anber
Testing Phase	Unit Testing

Table 5. TC-02: HX711 ADC Signal Conversion Stability

Test ID	TC-02
Test Name	HX711 ADC Signal Conversion Stability
Description	Verify that the HX711 analog-to-digital converter module produces stable, repeatable digital output when subjected to a constant known load, confirming that the 24-bit conversion is functioning correctly and the low-pass filter reduces noise.
Preconditions	Load cells connected to HX711 modules; constant known weight applied; Arduino Uno connected via USB; Serial Monitor open at 9600 baud.
Test Input	Constant static load of 200 g placed on a single load cell for 60 seconds.

Test Steps	1. Apply 200 g to Load Cell 1. 2. Record 100 consecutive readings from the Serial Monitor. 3. Compute mean, standard deviation, and min/max values. 4. Repeat with LPF enabled and disabled to compare noise levels.
Expected Result	Standard deviation of readings ≤ 1.0 g; all readings within $\pm 2\%$ of mean value; LPF-enabled readings show $\leq 50\%$ variance compared to raw readings.
Pass Criteria	Standard deviation ≤ 1.0 g and all readings within $\pm 2\%$ of expected weight.
Responsible	Yasin Anber
Testing Phase	Unit Testing

Table 6. TC-03: Tare and Zero-Point Reset Functionality

Test ID	TC-03
Test Name	Tare and Zero-Point Reset Functionality
Description	Verify that the system correctly resets the zero-point baseline (tare) upon initialization and that a deadzone of ± 1.0 g is enforced, preventing micro-fluctuations from being reported as real weight changes.
Preconditions	All load cells mounted on platform, no external load applied, system powered on.
Test Input	No external load; subsequently, a vibration-inducing tap of < 0.5 g equivalent is applied to the platform.
Test Steps	1. Power on system and allow tare to complete. 2. Read serial output with no load; verify all three sensors read 0.0 ± 1.0 g. 3. Apply a gentle tap; verify reading remains 0.0 g due to deadzone logic.
Expected Result	All three sensors report 0.0 g after tare. A sub-1 g disturbance does not alter the displayed weight.
Pass Criteria	Post-tare readings = 0.0 ± 1.0 g; sub-deadzone disturbance is suppressed.
Responsible	Selin Göç
Testing Phase	Unit Testing

Table 7. TC-04: CG Calculation Accuracy — Known Weight Distribution

Test ID	TC-04
----------------	-------

Test Name	CG Calculation Accuracy — Known Weight Distribution
Description	Validate that the CGCalculationModule correctly computes the X and Y coordinates of the center of gravity using moment equilibrium equations when a known, precisely measured weight is placed at a predetermined location on the aircraft platform.
Preconditions	All sensors calibrated (TC-01 passed), sensor coordinate matrix loaded with correct datum-relative positions, total weight > 10.0 g threshold to avoid divide-by-zero bypass.
Test Input	A 300 g weight placed at a known coordinate position (x = +5 cm, y = 0 cm) from the datum. Load cell readings fed to CG algorithm.
Test Steps	1. Place test weight at known platform coordinate. 2. Record F1, F2, F3 from the three load cells. 3. Manually compute expected CG_x and CG_y using moment equations. 4. Compare computed output from Arduino with manual calculation.
Expected Result	Computed CG_x and CG_y are within ± 3 mm of manually calculated expected CG position.
Pass Criteria	CG_x error $\leq \pm 3$ mm AND CG_y error $\leq \pm 3$ mm.
Responsible	Elif Sude Memiş, Selin Göç
Testing Phase	Unit Testing

Table 8. TC-05: CG Calculation — Divide by Zero Protection

Test ID	TC-05
Test Name	CG Calculation — Divide by Zero Protection
Description	Verify that the CGCalculationModule correctly bypasses CG calculation when total weight is below the 10.0 g threshold, preventing invalid division operations and erroneous output.
Preconditions	System initialized and tared. No load applied.
Test Input	Total weight = 0 g (no load) and total weight = 5 g (below threshold of 10 g).
Test Steps	1. Power on system with no load. 2. Observe serial output — CG calculation should be skipped. 3. Apply a sub-threshold load (< 10 g). 4. Verify that CG output is not transmitted and no divide-by-zero error occurs.
Expected Result	No CG coordinate is transmitted when total weight < 10.0 g. System remains in monitoring state.

Pass Criteria	Serial output does not include CG coordinate data when $W < 10$ g; no crash or exception.
Responsible	Elif Sude Memiş
Testing Phase	Unit Testing

Table 9. TC-06: Servo Motor PWM Control and Position Accuracy

Test ID	TC-06
Test Name	Servo Motor PWM Control and Position Accuracy
Description	Verify that each servo motor responds correctly to PWM control signals generated by the Arduino and physically moves the counterweight platform to the commanded position along the X and Y axes.
Preconditions	Servo motors connected to Arduino PWM output pins. Lead screw mechanism assembled and counterweight mounted. Homing sequence completed.
Test Input	Target position commands: $X = +10$ mm, $X = -10$ mm, $Y = +10$ mm, $Y = -10$ mm, and compound ($X = +8$ mm, $Y = +8$ mm).
Test Steps	1. Issue target position command from control module. 2. Observe physical displacement of counterweight platform using calibrated ruler. 3. Compare commanded position with measured physical displacement. 4. Repeat for all target positions.
Expected Result	Physical displacement matches commanded position within ± 1 mm for single-axis and ± 2 mm for compound movements.
Pass Criteria	All displacement measurements within stated tolerance; no mechanical binding observed.
Responsible	Yasin Anber, Sude İpekci
Testing Phase	Unit Testing

Table 10. TC-07: Counterweight Home Position Calibration

Test ID	TC-07
Test Name	Counterweight Home Position Calibration

Description	Verify that the MotorControlModule correctly executes the homing sequence upon system startup, driving the counterweight to the mechanical limit (endstop) to establish a reliable (0, 0) reference coordinate frame.
Preconditions	System powered on; mechanical endstops installed; counterweight displaced from home position.
Test Input	System startup command with counterweight at a random non-home position.
Test Steps	1. Place counterweight at a non-zero arbitrary position. 2. Power on system or issue homing command. 3. Observe motor movement toward endstops. 4. Verify system reports home position (0, 0) after homing completes.
Expected Result	Counterweight reaches endstop within 30 seconds. System logs currentMassPosition = (0.0, 0.0) after homing.
Pass Criteria	Homing completes without error; position reported as (0, 0).
Responsible	Yasin Anber
Testing Phase	Unit Testing

6.2 Integration Test Cases (TC-08 to TC-11)

Table 11. TC-08: Serial Communication — Arduino to Computer (UART)

Test ID	TC-08
Test Name	Serial Communication — Arduino to Computer (UART)
Description	Verify that the Arduino transmits formatted CG data packets to the computer over UART serial communication at 9600 baud and that data integrity is maintained, with no missing or corrupted packets.
Preconditions	Arduino connected to PC via USB. Python CGVisualizerApp running. Serial port configured at 9600 baud.
Test Input	Continuous system operation under a static 300 g load for 120 seconds.
Test Steps	1. Apply static 300 g load to platform. 2. Start CGVisualizerApp on PC. 3. Log all received serial packets for 120 seconds. 4. Analyze packet frequency, format, and integrity.

Expected Result	Data packets received in format 'weights:W g, X_CG: x, Y_CG: y' at a rate of at least 5 packets/second. Zero corrupted or malformed packets.
Pass Criteria	Packet rate ≥ 5 Hz; 0% packet corruption; data format matches specification.
Responsible	Selin Göç, Elif Sude Memiş
Testing Phase	Integration Testing

Table 12. TC-09: Sensor-to-Microcontroller Integration (HX711–Arduino)

Test ID	TC-09
Test Name	Sensor-to-Microcontroller Integration (HX711–Arduino)
Description	Verify the complete signal chain from load cell mechanical input through HX711 amplification and ADC conversion to Arduino digital processing, ensuring all three channels operate simultaneously without data loss or channel cross-talk.
Preconditions	All three load cells connected to respective HX711 modules. All HX711 modules connected to Arduino on designated DT/SCK pins (DT: 2,4,6; SCK: 3,5,7). Calibration completed.
Test Input	Simultaneous application of different known weights to all three load cells: 100 g (Front), 200 g (Right), 150 g (Left).
Test Steps	1. Apply specified weights simultaneously to all three load cells. 2. Record individual sensor readings from serial output. 3. Verify each reading corresponds to the correct load cell channel. 4. Verify total weight sums correctly.
Expected Result	Load Cell 1: 100 ± 2 g, Load Cell 2: 200 ± 4 g, Load Cell 3: 150 ± 3 g. Total weight: 450 ± 9 g. No channel cross-talk observed.
Pass Criteria	All three individual channel readings within $\pm 2\%$ tolerance; sum within $\pm 2\%$ of expected 450 g.
Responsible	Sude İpekci, Yasin Anber
Testing Phase	Integration Testing

Table 13. TC-10: CG Calculation to Balancing Mechanism Integration

Test ID	TC-10
----------------	-------

Test Name	CG Calculation to Balancing Mechanism Integration
Description	Verify that when the CGCalculationModule detects a CG deviation exceeding the balanceTolerance threshold, the ControlModule correctly computes a target counterweight position and the MotorControlModule physically drives the counterweight to that position.
Preconditions	Full system assembled. CG calculation module functional (TC-04 passed). Servo motors and lead screws operational (TC-06 passed). Serial communication active.
Test Input	An asymmetric load of 400 g placed off-center at (x = +7 cm, y = +3 cm) from the datum, creating a CG deviation exceeding the 2.0-unit tolerance threshold.
Test Steps	1. Apply asymmetric load. 2. Record computed CG deviation (Delta_X, Delta_Y). 3. Observe ControlModule output for target counterweight coordinates. 4. Measure actual physical displacement of counterweight. 5. Verify CG moves toward (0,0) after correction.
Expected Result	Target position computed correctly using mass-moment equation. Counterweight moves to calculated target within ± 2 mm. Post-correction CG deviation < balanceTolerance.
Pass Criteria	Post-correction CG error < 2.0 units; counterweight displacement within ± 2 mm of calculated target.
Responsible	Elif Sude Memiş, Selin Göç
Testing Phase	Integration Testing

Table 14. TC-11: CGVisualizerApp — Real-Time Display and Color Indicator

Test ID	TC-11
Test Name	CGVisualizerApp — Real-Time Display and Color Indicator
Description	Verify that the Python-based CGVisualizerApp correctly parses incoming serial data, updates the 2D graphical display with the real-time CG position marker, and correctly changes the marker color from red to green when the CG enters the tolerance zone.
Preconditions	Arduino transmitting serial data. CGVisualizerApp running on PC. Serial port correctly configured.
Test Input	First: a known imbalanced load resulting in CG error > ± 2.0 units. Then: manual adjustment to bring CG within ± 2.0 units of origin.

Test Steps	1. Launch CGVisualizerApp. 2. Apply imbalanced load; observe CG marker position and color. 3. Adjust load to bring CG within tolerance. 4. Verify color change and coordinate display accuracy.
Expected Result	Marker position on canvas corresponds to actual CG coordinates (scaled by factor of 25). Marker is red when CG error $> \pm 2.0$ and green when error $\leq \pm 2.0$.
Pass Criteria	Correct color transitions observed; displayed coordinates match serial data within rounding.
Responsible	Selin Göç
Testing Phase	Integration Testing

6.3 System Test Cases (TC-12 to TC-13, TC-17 to TC-18)

Table 15. TC-12: End-to-End System Functional Test — Full Balancing Cycle

Test ID	TC-12
Test Name	End-to-End System Functional Test — Full Balancing Cycle
Description	Validate the complete end-to-end system workflow: from load application and sensor measurement, through CG calculation, control decision, counterweight actuation, and real-time visualization — verifying that the system autonomously restores balance from a measurable imbalance.
Preconditions	Complete system assembled and operational. All prior unit and integration tests passed. System at home/tare position.
Test Input	An asymmetric aircraft model load (simulated using calibrated weights) creating CG deviations in both X and Y axes simultaneously.
Test Steps	1. Apply asymmetric load to aircraft model platform. 2. Allow system to measure, compute, and actuate without manual intervention. 3. Record time from load application to system-declared stable state. 4. Measure final CG deviation. 5. Document counterweight displacement.
Expected Result	System reaches stable, balanced state (CG within tolerance) within 30 seconds. No manual intervention required. System self-corrects.
Pass Criteria	CG error $<$ balanceTolerance after one correction cycle; time to balance ≤ 30 seconds.
Responsible	All team members

Testing Phase	System Testing
----------------------	----------------

Table 16. TC-13: Asymmetric Multi-Point Load Distribution Test

Test ID	TC-13
Test Name	Asymmetric Multi-Point Load Distribution Test
Description	Verify system correctness when all three load cells experience significantly different weight values, simulating realistic aircraft loading conditions such as uneven fuel, cargo, or passenger distribution.
Preconditions	System calibrated and functional. All sensors operational.
Test Input	Three different weights applied simultaneously: 500 g (Front), 150 g (Right), 300 g (Left), creating a complex CG offset.
Test Steps	1. Apply specified asymmetric loads to all three points. 2. Record CG_x and CG_y calculated by system. 3. Manually verify using moment formula. 4. Observe system response and counterweight movement. 5. Record final CG after balancing.
Expected Result	Computed CG matches manual calculation within ± 3 mm. System initiates balancing response. Final CG within acceptable tolerance.
Pass Criteria	CG calculation error $\leq \pm 3$ mm; system balancing response initiated within 5 seconds.
Responsible	Sude İpekci, Yasin Anber
Testing Phase	System Testing

Table 17. TC-17: Sensor Failure Detection and Safe State Transition

Test ID	TC-17
Test Name	Sensor Failure Detection and Safe State Transition
Description	Verify that the system correctly detects a sensor disconnection or anomalous reading and enters a safe state, preventing the issue from propagating to incorrect CG calculations or unsafe actuator commands.
Preconditions	System fully operational with all three load cells connected.
Test Input	Deliberate disconnection of one load cell's DT data line during active operation.

Test Steps	1. Start system with normal load. 2. Disconnect DT pin of one HX711 module. 3. Observe system behavior: serial output, actuator response. 4. Verify system either flags error or gracefully handles missing data. 5. Reconnect sensor and verify recovery.
Expected Result	System detects anomalous or missing sensor data within 5 seconds. Actuator motion stops or is constrained. System does not report false CG values. After reconnection, normal operation resumes.
Pass Criteria	Error detected within 5 s; no unsafe actuation; normal recovery after reconnection.
Responsible	Yasin Anber, Elif Sude Memiş
Testing Phase	System Testing

Table 18. TC-18: Mechanical Boundary Constraint Enforcement

Test ID	TC-18
Test Name	Mechanical Boundary Constraint Enforcement
Description	Verify that the MotorControlModule enforces the defined axis movement limits (axisLimits), preventing the counterweight from traveling beyond safe mechanical boundaries even when the ControlModule computes a target position outside those limits.
Preconditions	Motor control module configured with defined axisLimits. Counterweight at home (0,0) position.
Test Input	A commanded target position that exceeds the defined axisLimits in both X and Y axes.
Test Steps	1. Issue a command for counterweight position beyond axis limits. 2. Observe actual counterweight displacement. 3. Verify counterweight stops at boundary, not at commanded excessive position. 4. Verify no mechanical damage or motor stall.
Expected Result	Counterweight travels to the defined limit and stops. emergencyHalt() or limit-enforcement logic activates. No damage to mechanism.
Pass Criteria	Counterweight stops at boundary ± 1 mm; no mechanical binding or motor overload.
Responsible	Yasin Anber
Testing Phase	System Testing

6.4 Performance Test Cases (TC-14 to TC-16)

Table 19. TC-14: Control Loop Response Time

Test ID	TC-14
Test Name	Control Loop Response Time
Description	Measure the time elapsed from the moment a weight imbalance is introduced to the moment the system completes its correction cycle and reaches a stable balanced state, validating the real-time operation requirement.
Preconditions	System in balanced state. Stopwatch or software timer ready. Serial monitoring active.
Test Input	A sudden imbalance created by adding a 200 g weight off-center at $t = 0$.
Test Steps	1. Record timestamp when imbalance is applied. 2. Monitor serial output for stability detection event. 3. Record timestamp when system declares stable state. 4. Compute elapsed time. 5. Repeat 5 times and calculate average response time.
Expected Result	Average response time from imbalance detection to stable correction ≤ 15 seconds. No response time exceeds 30 seconds.
Pass Criteria	Mean response time ≤ 15 s; maximum ≤ 30 s across 5 trials.
Responsible	Elif Sude Memiş
Testing Phase	Performance Testing

Table 20. TC-15: Continuous Operation Stability Test (Endurance)

Test ID	TC-15
Test Name	Continuous Operation Stability Test (Endurance)
Description	Evaluate the long-term operational stability and reliability of the system under continuous running conditions to detect issues such as sensor drift, memory leaks, cumulative errors, or mechanical wear that may not appear in short-duration tests.
Preconditions	System fully assembled and operational. Consistent static load applied.
Test Input	A constant load of 350 g applied to the platform for a continuous 60-minute operation period.

Test Steps	1. Apply 350 g constant load. 2. Start system and begin logging all serial output. 3. Let system operate continuously for 60 minutes. 4. At 15-minute intervals, record CG coordinates and sensor values. 5. Analyze for drift or anomalies.
Expected Result	CG coordinates remain within ± 3 mm of initial calculated value over 60 minutes. No system resets, freezes, or communication errors. Sensor readings remain within $\pm 2\%$ throughout.
Pass Criteria	CG drift $\leq \pm 3$ mm over 60 minutes; zero unplanned system interruptions.
Responsible	All team members
Testing Phase	Performance Testing

Table 21. TC-16: System Response Under Dynamic Load Change

Test ID	TC-16
Test Name	System Response Under Dynamic Load Change
Description	Assess the system's ability to detect and respond correctly to dynamic changes in weight distribution — simulating real aircraft scenarios where load shifts occur during ground operations — and to re-achieve balance after each change.
Preconditions	System operational. Initial balanced state established.
Test Input	Sequential load changes: (1) add 200 g at off-center position, (2) remove 200 g and add 300 g at different position, (3) remove all additional loads.
Test Steps	1. Apply load change 1 and observe system response and final CG. 2. Apply load change 2 and repeat. 3. Remove all loads and verify system returns to near-zero CG. 4. Record response time for each transition.
Expected Result	System correctly detects each change, recomputes CG, and actuates counterweight to new balanced position within 30 seconds for each transition.
Pass Criteria	All three load transitions resolved to balanced state within 30 seconds each.
Responsible	Sude İpekci, Selin Göç
Testing Phase	Performance Testing

6.5 User Acceptance Test Case (TC-19)

Table 22. TC-19: User Interface Accuracy and Usability —UAT

Test ID	TC-19
Test Name	User Interface Accuracy and Usability — UAT
Description	Evaluate whether the CGVisualizerApp provides accurate, interpretable, and timely visual feedback to the operator, confirming that the user interface meets usability requirements for ground operations personnel.
Preconditions	CGVisualizerApp running. System operational. A non-expert user (simulated ground operator) briefed on basic system operation.
Test Input	A series of three different weight distributions applied sequentially by the test operator.
Test Steps	1. Operator applies load distribution 1 and interprets CG display. 2. Operator verifies whether CG indicator shows balanced (green) or imbalanced (red) state. 3. Repeat for load distributions 2 and 3. 4. Record operator feedback on display clarity and response speed.
Expected Result	Operator correctly identifies balance status in all three scenarios without assistance. CG coordinates update visibly within 2 seconds of load application.
Pass Criteria	100% correct balance status identification; display update latency ≤ 2 seconds.
Responsible	Selin Göç, Elif Sude Memiş
Testing Phase	User Acceptance Testing

6.6 Beta Test Case (TC-20)

Table 23. TC-20: Beta Test — Simulated Aircraft Pre-Flight CG Check

Test ID	TC-20
Test Name	Beta Test — Simulated Aircraft Pre-Flight CG Check
Description	Conduct a full simulated pre-flight CG verification scenario using the aircraft prototype model with realistic load distributions to evaluate the system's end-to-end performance in a near-operational environment, as would be conducted by aircraft maintenance personnel.

Preconditions	Aircraft prototype model mounted on the load cell platform. System fully operational. All prior tests passed. Test supervisor present.
Test Input	Simulated aircraft loading scenarios: (1) empty aircraft, (2) front-heavy load (simulating forward cargo), (3) rear-heavy load, (4) lateral imbalance.
Test Steps	1. Place aircraft model without additional load; verify baseline CG. 2. Apply each loading scenario sequentially. 3. Record system-computed CG for each. 4. Compare with theoretical expected CG for each scenario. 5. Verify automatic balancing response in scenarios 2–4.
Expected Result	Computed CG matches theoretical values within ± 5 mm for all four scenarios. Automatic balancing reduces CG error to within tolerance in scenarios 2–4. System operates without errors throughout.
Pass Criteria	CG accuracy $\leq \pm 5$ mm; auto-balancing effective in all three imbalanced scenarios; zero system errors.
Responsible	All team members
Testing Phase	Beta Testing

7. Test Schedule

Testing activities are scheduled over a four-week period in April 2026 is shown in Table 24, aligned with the CMPE 492 project submission and demonstration timeline. The schedule is structured to ensure that lower-level tests are completed and that sufficient time is allocated for defect resolution and re-testing between phases.

Table 24. Testing Schedule

Phase	Duration	Test Cases	Responsible	Dates (2026)
Unit Testing	3 days	TC-01 to TC-07	All members	07–09 Apr
Integration Testing	3 days	TC-08 to TC-11	Sude İpekci, Yasin Anber	10–12 Apr
System Testing	2 days	TC-12, TC-13, TC-17, TC-18	All members	14–15 Apr
Performance Testing	2 days	TC-14 to TC-16	Elif Sude Memiş, Selin Göç	16–17 Apr

Phase	Duration	Test Cases	Responsible	Dates (2026)
Defect Resolution & Retesting	2 days	Re-run of failed TCs	Developer on duty	18–19 Apr
User Acceptance Testing	1 day	TC-19	Selin Göç, Elif Sude Memiş	21 Apr
Beta Testing	2 days	TC-20	All members	22–23 Apr
Final Review & Documentation	2 days	All results review	All members	24–25 Apr

8. Roles and Responsibilities

Table 25 defines the roles and associated responsibilities of supervisor and all team members during the testing phase of the project.

Table 25. Roles and Responsibilities

Team Member	Role	Testing Responsibilities
Sude İpekci	Hardware Test Lead	Lead unit testing for load cell calibration (TC-01); support sensor-ADC integration testing (TC-09); coordinate hardware setup and teardown for all tests.
Yasin Anber	Embedded Systems Tester	Execute TC-02, TC-07, TC-09, TC-18; responsible for Arduino firmware-level debugging; validate HX711 communication protocol integrity.
Selin Göç	Software & UI Tester	Execute TC-03, TC-06, TC-08, TC-11, TC-16, TC-19; responsible for CGVisualizerApp testing and UART communication validation.
Elif Sude Memiş	Algorithm & Integration Tester	Execute TC-04, TC-05, TC-10, TC-14, TC-17; responsible for CG algorithm correctness and control loop integration verification.
All Team Members	System & Beta Testers	Jointly execute TC-12, TC-13, TC-15, TC-20; coordinate full-system setup for system and beta testing phases; compile final test results report.

Team Member	Role	Testing Responsibilities
Prof. Dr. Hakkı Gökhan İLK	Supervisor / Technical Reviewer	Review test plan; supervise beta testing phase; approve test completion criteria and sign off on final test report.

9. Control Procedures

The following control procedures govern the execution, documentation and management of all testing activities throughout the testing phase. Adherence to these procedures is mandatory for all team members to ensure consistency, traceability and quality of the testing process.

9.1 Test Execution Protocol

- All test cases must be executed in accordance with the procedure steps defined in Section 6. No ad hoc modifications to test procedures are permitted during execution without prior team approval.
- Each test case execution must be preceded by verification that all stated preconditions are met. A precondition checklist must be completed and signed before the test begins.
- All test results, both passing and failing, must be recorded in the Test Results Log immediately upon completion of each test case, including the date, tester name, observed outcome and pass/fail status.
- For any test case that results in a FAIL outcome, a defect report must be opened in the project defect tracking log within one hour of the test execution, documenting the observed behavior, the expected behavior and the test inputs used.

9.2 Defect Management

- Defects are classified by severity: Critical (system inoperable or unsafe), Major (primary feature not functioning), Minor (secondary feature degraded or cosmetic issue).
- Critical and Major defects must be resolved before advancing to the next testing phase. Minor defects may be deferred to the final review period at the discretion of the test lead.
- After a defect is resolved by the responsible developer, the affected test case must be re-executed in full to confirm the fix and close the defect report.
- All defect reports and their resolutions are retained as part of the project documentation archive.

9.3 Version Control and Configuration Management

- All firmware source code, Python application code, and test scripts are maintained under version control (Git). The exact firmware and software version under test must be recorded in the Test Results Log for each test session.
- Any change to hardware configuration (e.g., load cell replacement, calibration factor update) must be recorded in the hardware configuration log and the affected unit tests must be re-executed.
- The Test Plan document itself is version-controlled; any approved changes to the plan must be tracked with revision numbers and change descriptions.

10. Risks and Mitigation

Table 26 identifies the risks associated with the testing process, their potential impact on testing activities and outcomes, and the mitigation strategies adopted to reduce their likelihood or severity.

Table 26. Testing Risks

Risk	Likelihood	Impact	Mitigation Strategy
Sensor calibration drift between test sessions	Medium	High	Re-calibrate all load cells at the start of each test session; maintain calibration log with date-stamped calibration factors.
HX711 module noise or signal instability	Medium	High	Apply LPF with 0.05 Hz cutoff; verify filtering effectiveness in TC-02; replace module if instability persists.
Servo motor mechanical binding or slippage in lead screw	Low	High	Lubricate lead screw mechanism before each test session; verify homing sequence (TC-07) before executing actuation tests.
Arduino firmware crash or watchdog reset during continuous operation	Low	Critical	Implement watchdog timer in firmware; monitor serial output continuity during TC-15 endurance test; maintain firmware version log.
Serial port communication failure between Arduino and PC	Medium	Medium	Verify COM port assignment before each test session; use PySerial exception handling in CGVisualizerApp; test with alternative terminal if needed (TC-08).
Load cell physical damage during testing	Low	Critical	Do not exceed 80% of rated capacity (5 kg) during any test; handle load cell mounting with care; keep spare unit available.
Incorrect CG calculation due to uncorrected datum error	Low	Critical	Verify datum (0,0) origin alignment before system-level tests; cross-validate sensor coordinate matrix against physical measurements.
Time constraint: insufficient time to complete all test phases	Medium	Medium	Prioritize Critical and Major test cases; schedule parallel execution where possible; invoke defect deferral for Minor issues.
Beta test aircraft prototype not ready in time	Low	Medium	Begin beta test preparation in parallel with performance testing; use calibrated weight proxy in worst case.

11. Conclusion

This test plan provides a comprehensive framework for validating the functionality, performance, and reliability of the Aircraft Center of Gravity Detection and Balancing System. By clearly defining the scope, testing methodologies, test environment, responsibilities, and structured test cases, the plan ensures that all critical components of the system are thoroughly evaluated.

Through the application of unit, integration, system, and performance testing, the system is expected to achieve accurate CG calculations, stable sensor readings, and effective balancing operations. Additionally, user acceptance and beta testing contribute to verifying that the system meets both technical requirements and practical usability expectations.

The structured testing process outlined in this report minimizes potential risks, improves system robustness, and supports the development of a safe and efficient solution for aircraft balance management. Overall, this test plan serves as a guideline to ensure that the final system meets quality standards, operates reliably under different conditions, and fulfills its intended purpose successfully.

12. References

- [1] Delta Motion, “Load cell fundamentals,” Delta Computer Systems, Inc. [Online]. Available: https://deltamotion.com/support/webhelp/rmctools/Content/Controller_Features/Transducer_Basics/Load_Cell_Fundamentals.htm.
- [2] M. H. M. Diah, “Advancing strain gauge technology: Design, integration, and industrial applications,” *Journal of the TSSA*, no. 230, p. 144, 2025. [Online]. Available: <https://tssa.my/index.php/jtssa/article/view/230/144>.
- [3] AVIA Semiconductor, “HX711: 24-Bit Analog-to-Digital Converter (ADC) for weigh scales,” Datasheet, SparkFun Electronics. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf.
- [4] R. Santos and S. Santos, “Arduino with load cell and HX711 amplifier (digital scale),” *Random Nerd Tutorials*. [Online]. Available: <https://randomnerdtutorials.com/arduino-load-cell-hx711/>.
- [5] Bitfoic, “Explore HX711 – 24-bit ADC: Pinout, pin configuration, datasheet,” Sep. 15, 2023. [Online]. Available: <https://www.bitfoic.com/components/explore-hx711%E2%80%9324-bit-adc-pinout-pin-configuration-datasheet?id=117>.
- [6] N. A. Latha and B. R. Murthy, “Arduino based weighting scale using load cell,” *International Journal of Scholarly Research in Science and Technology*, vol. 3, no. 6, p. 704, 2017. [Online]. Available: <https://ijsrst.com/paper/2443.pdf>.
- [7] Arduino, “UNO R3 – Arduino UNO is a microcontroller board based on the ATmega328P.” [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3>.
- [8] Raptor Scientific, “Center of Gravity Explained.” [Online]. Available: <https://raptor-scientific.com/resources/center-of-gravity/>.
- [9] “Moment calculations,” Southern Crescent Technical College. [Online]. Available: <https://www.sctech.edu/wp-content/uploads/TigerNet%20Docs/Adult%20Ed/MS%20Science%20Moment%20Calculations.pdf>.
- [10] “Center of Gravity,” *Engineering Statics*. [Online]. Available: https://engineeringstatics.org/Chapter_07-center-of-gravity.html.
- [11] IEEE Std 829-2008. *IEEE Standard for Software and System Test Documentation*. IEEE, 2008.
- [12] IEEE Std 1012-2016. *IEEE Standard for System, Software, and Hardware Verification and Validation*. IEEE, 2016.